

$$F = G \frac{m_1 m_2}{d^2}$$

$$i\hbar \frac{\partial}{\partial t} \psi = \hat{H} \psi$$

$$\phi(x) = \frac{1}{\sqrt{2\pi\sigma}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$F - E + V = 2$$

BOOSTING ALGORITHMS

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$

$$\frac{df}{dt} = \lim_{h \rightarrow 0} \frac{f(t+h) - f(t)}{h}$$

TYPES OF BOOSTING ALGORITHMS

- There are three boosting algorithms
- * Adaboost algorithm
- * Gradient Boost algorithm
- * XGBoost algorithm



1. AdaBoost (Adaptive Boosting)

Main idea:

- * Combine many weak learners model usually Decision Tree with depth = 1 nothing but node with 2 leaf and make strong model.

- * Each new model focuses more on the mistakes or incorreced prediction value made by previous model.

- * And it adjusts weights by correctly predicted reduce the weight and incorrectly predicted increase the weight and pass it to next model there by next model give more importance to incorrectly predicted row. So hard to predict points get more attention.

How it works:

Start with equal weights for all training samples.

Train the first weak learner.

Check which points are predicted incorrectly

Increase weight for the misclassified and mispredicted points.

Train next weak learner model on this weighted data.

Repeat until the set number of learners is reached.

Combine all learners giving more importance weight to ones with better performance.

Key points:

Work for both
regression and
classification.

Work well on clean data
but sensitive to noise
& Outliers

Work exponential weight
updates based on
prediction error.

2. Gradient Boosting (GBM):

Main Idea:

- * Builds tree sequentially just like AdaBoost.

- * Instead of adjusting weights, it fits the new model to the residual errors of the previous model using gradient decent.

- * More flexible work with various loss functions not just exponential loss.

Key Points:

Tries to minimize a loss function step by step.

SLOWER than
Adaboost, but
usually more
accurate.

Handle Outliers
better.

XGBoost (Extreme Gradient Boosting):

It work by extra improvement over Gradient Boosting.

Regularization: Adds L1 and L2 penalties to avoid overfitting.

Paraller Processing: Builds tree much faster.

Sparse data Handling: Handle missing values smartly.

Tree Pruning: stop splitting when further splits don't improve the model significantly.

Column subsampling: Uses only a randm subset of features for each tree like random forest.

Key Points:

Much faster and more accurate for large datasets.

Works for regression and classification

Has many Hyperparameters to tune.



THANK YOU